

Cognitive Robotics

Studienarbeit T3200

Studiengang Elektrotechnik
An der Dualen Hochschule Stuttgart
Campus Horb

Eingereicht von:

Name:	Luis Magnus Thiel	Adrian Sommer
Matrikelnr.:	8616207	9519924
Straße, Nr.:	Hugo-Bertsch-Straße 15	Hechinger Straße 40
PLZ, Ort:	72459 Albstadt	72336 Balingen

Partner:	ASSA ABLOY Sicherheitstechnik GmbH	Groz-Beckert KG
Adresse:	Bildstockstraße 20	Parkweg 2
Ort:	72458 Albstadt	72458 Albstadt

Studienjahrgang: TET2023

Bearbeitungszeitraum: 02. März 2026 bis 22. Juni 2026

Betreuer: Norman Bläse (Mercedes-Benz)

Kurzzusammenfassung

Diese Studienarbeit befasst sich mit der Optimierung und Erweiterung eines autonomen Robotersystems, bestehend aus einem Mitsubishi Movemaster EX RV-M1, einer 3D-ToF-Kamera sowie einer 2D-Industriekamera. Dabei ist das übergeordnete Ziel die Transformation des vorliegenden Aufbaus hin zu einem intelligenten System, welches durch die Implementierung von bildbasierten Sicherheitsfunktionen Potenziale für einen kollaborativen Betrieb aufzeigt.

Zunächst wird der Einfluss verschiedener Beleuchtungsbedingungen auf das Bildgewinnungssystem in einer Messreihe erforscht und mögliche Optimierungen erarbeitet. Nachfolgend gibt die Analyse der C++/C#-Softwarearchitektur Aufschluss über vorhandene Schwachstellen, wie auftretende Probleme bei der Thread-Synchronisierung und der Implementierung einer statischen Robotersteuerung. Für letzteres wird durch die Konzeptionierung einer dynamischen und geschlossenen Ansteuerung eine Alternative geschaffen.

Aufbauend auf den Erkenntnissen der Analyse und Konzeptionierung der dynamischen Ansteuerung wird ein Gesamtkonzept für die Entwicklung einer *Geschwindigkeits- und Abstandsüberwachung (SSM)* erstellt. Dieses Konzept nutzt die Fusion der 2D- und 3D-Bilddaten, um dynamische Warn- und Stoppbereiche softwareseitig zu realisieren. Die Arbeit bildet somit das Fundament für die sicherheitstechnische und steuerungstechnische Implementierung in der nachfolgenden Studienarbeit.

Abstract

This student research project deals with the optimisation and expansion of an autonomous robot system consisting of a Mitsubishi Movemaster EX RV-M1, a 3D ToF camera and a 2D industrial camera. The overarching goal is to transform the existing setup into an intelligent system that reveals the potential for collaborative operation through the implementation of image-based safety functions.

First, the influence of different lighting conditions on the image acquisition system is investigated in a series of measurements and possible optimisations are developed. Subsequently, the analysis of the C++/C# software architecture highlights existing weaknesses, such as problems occurring in thread synchronisation and the implementation of a static robot control system. For the latter, an alternative is created by designing a dynamic and closed-loop control system.

Based on the findings of the analysis and design of the dynamic control system, an overall concept for the development of a *Speed and Separation Monitoring (SSM)* system is created. This concept uses the fusion of 2D and 3D image data to implement dynamic warning and stop areas software-side. The work thus forms the foundation for the safety-related and control-related implementation in the subsequent student research project.

Selbständigkeitserklärung

Hiermit erklären wir, Luis Magnus Thiel und Adrian Sommer, dass wir diese schriftliche Studienarbeit selbständig verfasst haben. Es wurden keine anderen als die angegebenen Hilfsmittel und Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen sind als solche gekennzeichnet.

Horb, den 15. Juni 2026

Ort, Datum



Unterschrift (Luis Magnus Thiel)

Horb, den 15. Juni 2026

Ort, Datum



Unterschrift (Adrian Sommer)

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	2
2.1	Stand der Technik: Safety-Implementation & Retrofitting .	2
2.2	Bildverarbeitung	4
2.3	Grundlagen der Informatik	4
2.3.1	Producer-Consumer-Pattern	4
2.3.2	Task Parallel Library (TPL)	4
2.3.2.1	Task-based Asynchronous Pattern (TAP) .	5
2.3.2.2	Task Cancellation/kooperativer Abbruch .	6
2.3.3	Vergleich: Polling & reaktives System	6
2.3.4	TCP-Socket & Protokoll-Parsing	6
3	Ausgangssituation/IST-Analyse	7
3.1	Architekturlimitationen des bisherigen Systems	7
3.1.1	Sequenzielle Befehlsstruktur	7
3.1.1.1	Theoretische Limitation	8
3.1.1.2	Messexperiment	9
3.1.1.3	Praktische Limitation - Laufzeitmessung mit Simulationsexperiment	9
4	Architekturoptimierung und -erweiterung	10
4.1	Befehlswarteschlange für dynamischere Robotersteuerung	10
5	Implementation der Erweiterungen	11
5.1	Safetyimplementation	11
5.1.1	Kriterienbasierte Evaluierung der Sicherheitsüber- wachungssysteme	11
5.1.2	Gegenüberstellung spezialisiertes KI-Modell und einfacher Objekterkennungsalgorithmus	15
6	Fazit & Ausblick	16
7	Anhang	17
	Literaturverzeichnis	vii
	Abkürzungsverzeichnis	vii

Abbildungsverzeichnis	x
Erklärungen zur Arbeit und Hilfsmitteln	xi

1 Einleitung

Die vorliegende Studienarbeit befasst sich im ersten Schritt mit der fundierten Erarbeitung der verwendeten methodischen und technologischen Grundlagen. Darauf aufbauend erfolgt eine detaillierte Analyse der Ausgangslage, welche explizit an die Ergebnisse und den Stand der vorherigen Studienarbeit anknüpft. Bestandteil dieser Evaluation sind eine umfassende Ist- und Laufzeitanalyse sowie die Ausarbeitung daraus abgeleiteter Optimierungsvorschläge.

Auf Basis dieser Erkenntnisse widmet sich die Arbeit der Konzeptionierung neuer Lösungsansätze. Ein zentraler Fokus liegt hierbei auf der Optimierung und Erweiterung der bestehenden Systemarchitektur. Dies beinhaltet insbesondere die effiziente Speicherung und Instanziierung von Punktwolken. Zur erweiterten visuellen Auswertung wird eine Funktion zur wahlweisen einfachen oder doppelten Anzeige der Punktwolken integriert, wobei der Wechsel zwischen diesen beiden Darstellungsmodi nutzerfreundlich über eine neu implementierte Auswahl Taste erfolgt.

Ein weiterer inhaltlicher Schwerpunkt ist die informationstechnische Umsetzung einer Befehlspipeline sowie relevanter Safety-Features. In diesem Zuge werden die zugrunde liegende Softwarestruktur und die angewandten Design-Patterns detailliert beleuchtet. Nach einer kritischen Abwägung und Einordnung verschiedener Ansätze zur Safety-Implementierung wird die Wahl der finalen Methode sachlich begründet und deren praktische Umsetzung dargelegt. Den Abschluss der Arbeit bildet ein Ausblick, der potenzielle Erweiterungen, gezielte Verbesserungsvorschläge und konkrete Anknüpfungspunkte für nachfolgende Studienarbeiten aufzeigt.

2 Grundlagen

2.1 Stand der Technik: Safety-Implementation & Retrofitting

Für die Modernisierung älterer Industrie- und Robotersysteme (Retrofitting) im Hinblick auf aktuelle Sicherheitsstandards und moderne Sensortechnik existieren vielfältige technologische Ansätze. Diese lassen sich anhand verschiedener Systemparameter kategorisieren. Ein grundlegendes Unterscheidungsmerkmal ist die Dimensionierung der Bilderfassung, bei der entweder kostengünstige 2D-Kameras oder tiefenauflösende 3D-Kamerasysteme eingesetzt werden. Zudem variiert die Systemarchitektur hinsichtlich der Kameraanzahl: Während Monokamerasysteme eine einfache Raumüberwachung ermöglichen, bieten Multikamerasysteme durch hardwareseitige Redundanz eine deutlich verlässlichere Objekterkennung sowie Schutz vor Verdeckungen im sensornahen Bereich.

Die algorithmische Auswertung lässt sich wiederum in zwei Philosophieansätze unterteilen: die gezielte Klassifizierung spezifischer Objekte wie menschlicher Gliedmaßen oder die universelle Erfassung jeglicher Fremdobjekte innerhalb eines definierten Raumbereichs. Ergänzend oder alternativ zu visuellen Kamerasystemen kommen in der Praxis berührungslose Sensoren auf kapazitiver, induktiver oder laserbasierter Basis (z.B. Laserscanner) zum Einsatz.

Die informationstechnische Absicherung des Arbeitsraumes erfolgt je nach Systemkomplexität über die Festlegung einer einzigen, starren Verbotszone oder über die Aufteilung in mehrere, flexible Zonen. Letzteres ermöglicht die Implementierung einer dynamischen Geschwindigkeitsregelung (Dynamic Speed Control), bei der die Systemgeschwindigkeit adaptiv an die Position des Menschen angepasst wird. Diese Verbots- und Schutzräume können entweder statisch im Raum verankert oder dynamisch an die aktuelle Kinematik sowie den Bremsweg der Anlage gekoppelt werden, wie es unter anderem in Konzepten des Fraunhofer-Instituts [1] realisiert wird.

Ein zentrales wissenschaftliches Fundament für solche flexiblen Sicherheitsarchitekturen beschreiben Bdiwi, M., Pfeifer,

M. und Sterzing, A. [2], die sich intensiv mit unterschiedlichen Aufgabenszenarien der Mensch-Roboter-Kollaboration (MRK) befassen. Die Autoren schlagen eine systematische Taxonomie vor, welche die Interaktion zwischen Mensch und Maschine in vier aufeinander aufbauende Stufen gliedert:

**Level 1: Geteilter Raum
(Sicherheitsgerichteter Stopp)**

**Level 2: Gemeinsame Aufgabe
(Speed & Separation Monitoring)**

**Level 3: Handing-Over
(Hybride Steuerung / Dynamik)**

**Level 4: Physische Interaktion
(Kraftregelung / Guiding)**

- **1. Geteilter Arbeitsraum:** In diesem Szenario arbeiten Mensch und Roboter räumlich nah beieinander, führen jedoch strikt getrennte Aufgaben in exakt zugewiesenen Zonen aus. Sobald der Mensch die virtuelle Grenze zur Roboterzone überschreitet, detektiert die Sensorik eine Schutzraumverletzung und löst unverzüglich einen Sicherheitsstopp der Anlage aus.
- **2. Gemeinsame Aufgabe ohne physischen Kontakt:** Hier agieren beide Akteure innerhalb einer vordefinierten Kooperationszone an einem gemeinsamen Prozess. Anstelle eines abrupten Halts wird die Geschwindigkeit des Roboters in Abhängigkeit vom realen Abstand zum Menschen kontinuierlich gedrosselt.
- **3. Bauteilübergabe (*Handing-over*):** Dieses Level beschreibt den direkten Transfer von Werkzeugen oder Werkstücken zwischen Mensch und Roboter. Eine hybride Steuerung sorgt in dieser Phase dafür, dass der Manipulator dynamisch und feinfühlig auf die Handbewegungen des Menschen im freien Raum reagiert.
- **4. Gemeinsame Aufgabe mit physischer Interaktion:** Auf der höchsten Kooperationsstufe ist ein direkter Körperkontakt zwingend erforderlich. Ein Praxisbeispiel hierfür ist die manuelle Führung eines Schwerlastroboters durch menschliche Handkräfte, um schwere Bauteile präzise zu positionieren.

Die technische Umsetzung der Überwachung innerhalb dieser Interaktionsstufen stützt sich im Wesentlichen auf drei Säulen: die permanente Überwachung der internen Roboterparameter (wie Position, Achsgeschwindigkeiten und Kräfte), eine automatisierte Ereigniserkennung zur Identifikation kritischer Zustände (z. B. die visuelle Verdeckung eines Menschen im Sensorschatten) sowie eine kamerabasierte Nahfelderkennung. Letztere dient dazu, im direkten Umfeld des Endeffektors sowohl die sensorische Bereitschaft des Bedieners (z. B. via Gesichtserkennung) als auch die exakte Lage von Körperteilen zu erfassen, um Verletzungen oder Quetschungen effektiv zu verhindern.

2.2 Bildverarbeitung

Filter, Algorithmen, Linien-/Kantendetektion mit OpenCV

2.3 Grundlagen der Informatik

2.3.1 Producer-Consumer-Pattern

2.3.2 Task Parallel Library (TPL)

Die *Task Parallel Library (TPL)* ist eine .NET-Bibliothek, die es dem Programmierer vereinfacht, Parallelität und Nebenläufigkeit in C#-Programmen umzusetzen [3]. Für diese Arbeit ist vor allem das Konzept der Nebenläufigkeit (*engl.: concurrency*) relevant. Darunter versteht man allgemein, dass "mehr als nur eine Sache zur gleichen Zeit gemacht wird" [4, Kapitel 1]. In der praktischen Anwendung ermöglicht das eine Entkopplung von Rechenoperationen oder Ein-/Ausgabeoperationen vom primären Arbeitsstrang einer Anwendung. In Abbildung 2.1 werden synchrone und asynchrone

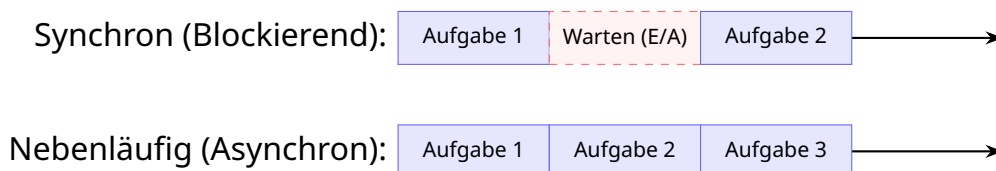


Abbildung 2.1: Problematik von blockierenden Abläufen

Abläufe gegenübergestellt. Es lässt sich erkennen, dass bei einem

synchronen Ablauf eine Wartezeit den aufrufenden Thread blockiert - TPL löst diese Problematik durch die Einführung von asynchronen *Tasks*.

Nebenläufigkeit vs. Parallelität

Um die Rolle der TPL im Kontext dieser Arbeit zu präzisieren, muss zwischen Nebenläufigkeit und Parallelität unterschieden werden. Während Parallelität die gleichzeitige Ausführung mehrerer Aufgaben auf verschiedenen Prozessorkernen beschreibt, definiert Nebenläufigkeit die organisatorische Strukturierung eines Programms. Ein nebenläufiges System ist darauf ausgelegt, Aufgaben durch verschachtelten Fortschritt (*engl.: Interleaving*) so zu bearbeiten, dass eine Anwendung reaktionsfähig bleibt, während sie im Hintergrund andere Prozesse abarbeitet.

Dabei ist Multithreading – also die Verwendung mehrerer Ausführungsstränge – lediglich eine Form der nebenläufigen Programmierung. Moderne Frameworks wie die TPL bieten hierfür übergeordnete Abstraktionen, die den direkten, fehleranfälligen Umgang mit Threads durch effizientere Konzepte ersetzen.

Task-Modell

Die TPL löst die starre Bindung an Betriebssystem-Threads durch das Konzept des Tasks (.NET-Bibliothek `System.Threading.Tasks`) auf. Ein Task repräsentiert eine asynchrone Operation und verkörpert das Entwurfsmuster eines Futures oder Promises – ein logisches Versprechen auf ein in der Zukunft liegendes Ergebnis. Anstatt Aufgaben fest an schwerfällige Betriebssystem-Threads zu binden, entkoppelt die TPL die logische Aufgabe von der physikalischen Ausführung. Ein interner Thread-Pool verwaltet eine minimale Anzahl von Arbeiter-Threads und verteilt die anfallenden Tasks dynamisch. Wartet ein Task beispielsweise auf eine Netzwerkanfrage, gibt er den physischen Thread sofort für andere nebenläufige Aufgaben frei.

2.3.2.1 Task-based Asynchronous Pattern (TAP)

Das Task-based Asynchronous Pattern (TAP) setzt das zuvor beschriebene Task-Modell ab .NET-Framework 4 als Standard für die asynchrone Programmierung in C# durch [5]. Hierzu kommen die Schlüsselwörter **async** und **await** zum Einsatz.

2.3.2.2 Task Cancellation/kooperativer Abbruch

2.3.3 Vergleich: Polling & reaktives System

2.3.4 TCP-Socket & Protokoll-Parsing

3 Ausgangssituation/IST-Analyse

3.1 Architekturlimitationen des bisherigen Systems

3.1.1 Sequenzielle Befehlsstruktur

Die vorliegende Befehlsübermittlung an den Movemaster EX RV-M1 wird in der Software *Objekterkennung_RoboETH* sequenziell realisiert. 3.1 zeigt qualitativ auf, wie eine solche Abfolge aussieht. Es ist

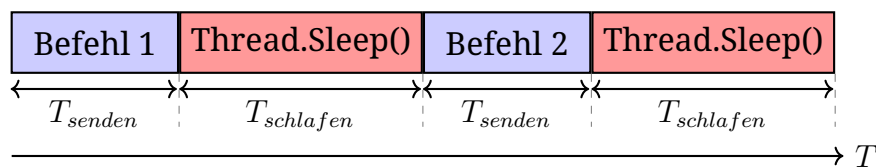


Abbildung 3.1: Sequenzielle Abfolge an Befehlen (Eigene Darstellung)

erkennbar, dass nach jedem gesendeten Befehl ein *Thread.Sleep()-Befehl* den gesamten Hauptthread für eine nicht vernachlässigbare Zeitspanne T_{schlafen} blockiert. Die theoretische Zeit T die benötigt wird, bis ein Befehl vollständig an den Roboter gesendet, von ihm verarbeitet und ausgeführt wird, lässt sich vereinfacht mit $T_{\text{befehlsausfuehrung}} = T_{\text{senden}} + T_{\text{schlafen}}$ beschreiben.

Während dieser Zeitspanne friert die gesamte Benutzeroberfläche sowie die Handlungsfähigkeit des Hauptthreads ein. Um die Auswirkungen der sequenziellen Befehlsbereitstellung auf die Gesamtlaufzeit und die Systemstabilität zu quantifizieren, ist eine detaillierte Analyse der Ausführungszeiten sowie der Thread-Verteilung erforderlich. Hierfür wird ein reproduzierbares Messexperiment aufgesetzt - hierzu nachfolgend mehr.

Das Ziel dieser Analyse ist die Identifikation der kritischen Pfade. Durch eine explizite Messung der Zeitstempel für den Eintritt in die Befehlsverarbeitung, die Dauer der Hardware-Kommunikation und die Verweildauer in den Wartezuständen wird der „Flaschenhals“

der synchronen Architektur isoliert. Erst durch diese messtechnische Herleitung wird transparent, an welcher Stelle die CPU-Last ungenutzt bleibt und wie sich die Thread-Auslastung sowie die Laufzeit bei einer Umstellung auf eine asynchrone Pipeline-Architektur verbessern lässt.

3.1.1.1 Theoretische Limitation

Wird erneut 3.1 betrachtet, so fällt auf, dass die Zeit, welche benötigt wird, um die Koordinaten der einzelnen Objekte im Arbeitsbereich des Roboters zu berechnen, nicht eingerechnet wird. Grund hierfür ist die Annahme, dass die Berechnungszeit in beiden Fällen (sequenzielle Befehlsstruktur oder asynchrone Befehlspipeline) voraussichtlich gleich ist.

Die theoretische Gesamtlaufzeit T_s einer Befehlssequenz lässt sich durch Summation der Befehlsausführungszeiten $T_{befehlsausfuehrung}$ bestimmen:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k T_{befehlsausfuehrung,i,j} \quad (3.1)$$

Einsetzen der zuvor beschriebenen Zeiten ergibt:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.2)$$

Hierbei repräsentiert der Index i das jeweilige Objekt innerhalb der Bearbeitungsreihenfolge (mit n als Gesamtzahl der Objekte), während der Index j die spezifische Befehlsposition innerhalb eines Manipulationszyklus angibt (mit k als Anzahl der Befehle pro Objekt).

In den Gleichungen 3.1 und 3.2 lässt sich die direkte Korrelation zwischen dem Parameter n und der durch die Bildverarbeitungsklasse erkannten Objekte erkennen. Des weiteren wird verdeutlicht, dass die Gesamtausführungszeit einer Befehlssequenz (bestehend aus $1, \dots, k$ Befehlen pro Objekt) linear von der Objektanzahl n und der aufsummierten Wartezeiten durch *Thread.Sleep()*-Befehlen abhängt.

Bei dieser theoretischen Betrachtung wird bewusst eine Worst-Case-Abschätzung mittels der fest definierten Wartezeiten getroffen, da der vorliegende Roboter und die zugehörige Software keine Bestätigung für eine abgeschlossene Bewegung liefern. Das wird in der Software durch statische Wartezeiten (erfahrungsbasiert oder alternativ grob

berechnet) kompensiert, was in der Praxis allerdings zu Einbußen der Systemeffizienz führen kann.

3.1.1.2 Messexperiment

Um die theoretisch hergeleiteten Ineffizienzen und die zeitlichen Auswirkungen der sequenziellen Befehlerteilung quantitativ nachzuweisen, wurde ein deterministisches Messexperiment aufgesetzt. Selbiges muss für Vergleichbarkeit und Bewertbarkeit auch bei nachfolgenden Erweiterungen des Systems zu Grunde gelegt werden.

3.1.1.3 Praktische Limitation - Laufzeitmessung mit Simulations-experiment

Objekt-ID	Theoretische Zeit T_S [s]	Gemessene Zeit T_{mess} [s]
Objekt 1	17,5	
Objekt 2	17,5	
Objekt 3	17,5	
Objekt 4	17,5	
Objekt 5	17,5	
Mittelwert	17,5	
Gesamtzeit		

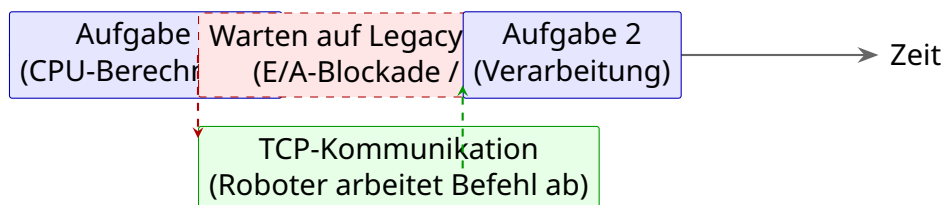
Tabelle 3.1: Vergleich der theoretischen Laufzeit T_S mit den gemessenen Werten T_{mess}

4 Architekturoptimierung und -erweiterung

4.1 Befehlswarteschlange für dynamischere Robotersteuerung

Synchroner Ablauf (Blockierend):

Hauptthread ist während der E/A-Operation blockiert



Asynchroner Ablauf (Nebenläufig via TPL / TAP):

Thread-Freigabe bei E/A-Operationen dank async/await

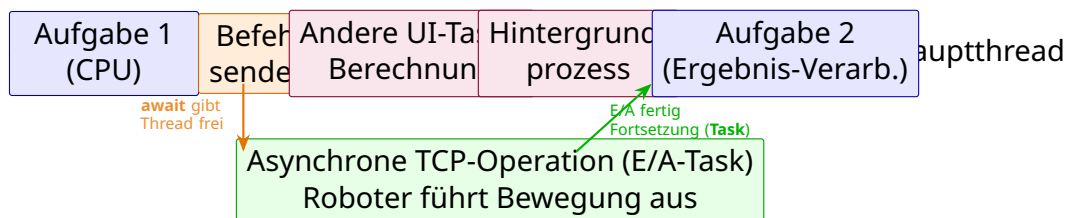


Abbildung 4.1: Gegenüberstellung von blockierenden synchronen Abläufen und dem ressourceneffizienten, asynchronen Task-basierten Muster (TAP) bei E/A-Operationen.

5 Implementation der Erweiterungen

Warteschlange, Safety, Punktwolkenspeicherung, ...

5.1 Safetyimplementation

5.1.1 Kriterienbasierte Evaluierung der Sicherheitsüberwachungssysteme

In der modernen industriellen Produktion hat sich eine Vorgehensweise etabliert, die auf einer systematischen Taxonomie der Aufgaben und Anwendungsfelder basiert. Dieser Ansatz ist maßgeblich, um funktionierende sowie finanziell und funktionell optimierte Absicherungssysteme zu entwickeln. Die Interaktion zwischen Mensch und Maschine wird in diesem Kontext standardmäßig als Human-Robot Interaction (HRI) beziehungsweise im industriellen Umfeld als Mensch-Roboter-Kollaboration (MRK) abgekürzt.

Die Auswahl einer geeigneten MRK-Sicherheitsüberwachung stützt sich auf diverse technologische und funktionale Entscheidungskriterien. Diese werden in der folgenden Evaluierungsmatrix anhand zentraler Kategorien – wie dem Aufbau des Arbeitsbereichs, dem den Notzustand einleitenden Ereignis, der eingesetzten Sensorik sowie der resultierenden Systemreaktion des Roboters – gegenübergestellt. Um den Rahmen dieser Arbeit zu wahren, konzentriert sich die Darstellung für jede Kategorie exemplarisch auf eine Gegenüberstellung zweier primärer Ausprägungen (Variante A und Variante B), wenngleich in der Praxis weitere technologische Mischformen existieren.

Für die informationstechnische Abgrenzung des Kollaborationsraumes stehen sich statische und dynamische Konzepte gegenüber. Ein statischer Sicherheitsbereich zeichnet sich primär durch die Einfachheit seiner Umsetzung aus. Die Begrenzung kann entweder über einen physischen, kontrastreichen Rahmen im realen Hintergrund des Arbeitsbereichs oder rein softwareseitig über die Definition eines virtuellen Rahmens (Region of Interest, ROI) innerhalb der Bildmatrix realisiert werden. Eine Kaskadierung des Raumes in mehrere Teilzonen ist zudem möglich und stellt das grundlegende Fundament für die

Kriterium	Variante A	Variante B
Sicherheitsbereich	statisch	dynamisch
Ereignis	Objekterkennung	Handerkennung
Sensorik	physisch	visuell
Aktion	Nothalt	Speed and Separation-Prinzip

Tabelle 5.1: Entscheidungsmatrix zur Auswahl der MRK-Sicherheitsüberwachung

Implementierung des Speed and Separation Monitoring (SSM) dar. Die für diese Studienarbeit relevanteste Methode nutzt das definierte Blickfeld (Field of View, FoV) einer 2D-Kamera als feste Begrenzungsebene. Erkennt der Algorithmus innerhalb dieses Sichtfeldes eine Verletzung der definierten Grenzen durch eine Bedrohung, wird die Sicherheitsreaktion des Roboters initiiert. Da der Sichtbereich der Kamera hierbei als absolute Grenze fungiert, spielt die exakte geometrische Ausrichtung auf den zu betrachtenden Arbeitsraum eine prozesskritische Rolle.

Demgegenüber passt sich ein dynamischer Sicherheitsbereich adaptiv in Echtzeit an die aktuelle Pose des Roboterarms an. Die Komplexität dieser Implementierung ist bedeutend höher als bei statischen Systemen, da für die Hüllkurvenberechnung permanent die Vorwärtskinematik des Roboters gelöst und die Transformation der Koordinatensysteme in den dreidimensionalen Raum berechnet werden müssen. Dies bedingt eine sehr geringe Datenbus-Latenz (Echtzeitfähigkeit), stellt hohe Anforderungen an die Rechenleistung des Gesamtsystems und erfordert für den Werker eine dynamische, visuelle Anzeige des aktiven Bereichs, beispielsweise über Lichtprojektionen oder innerhalb eines Digitalen Zwillings. Der technologische Mehraufwand resultiert jedoch in einer maximalen Interaktionsfreiheit zwischen Mensch und Roboter und bietet eine hervorragende Kombinierbarkeit mit dem SSM-Prinzip.

Bei der algorithmischen Ereignissteuerung wird zwischen universeller Objekterkennung und spezifischer Handerkennung unterschieden. Dient eine generelle Objekterkennung als auslösendes Ereignis für einen Sicherheitsstopp oder eine Geschwindigkeitsreduzierung, basiert dies meist auf klassischer Kantenextraktion oder Hintergrundsubtraktion. Sobald ein Fremdobjekt im Arbeitsbereich detektiert wird, hält die Anlage an. Ein inhärentes Problem dieses Ansatzes liegt darin, dass der Roboterarm selbst als Objekt erfasst

wird und permanent aus dem Bild herauskalkuliert (maskiert) werden muss, was den Programmieraufwand erhöht. Der wesentliche Vorteil liegt in der einfachen grundlegenden Implementation. Als kritischer Nachteil erweist sich jedoch die hohe Fehlerwahrscheinlichkeit, da transiente Umgebungsstörungen fälschlicherweise als Pseudoobjekte erkannt werden. Diese hohe Rate an Falsch-Positiv-Detektionen führt zu häufigen unplanmäßigen Anlagenstillständen, was gemäß VDI-Richtlinie 3634 die wirtschaftliche Gesamtanlageneffizienz (Overall Equipment Effectiveness, OEE) massiv senkt.

Eine selektivere Alternative stellt die ereignisbasierte Handerkennung dar. Hierbei detektiert ein spezielles, auf die Erkennung menschlicher Hände trainiertes KI-Modell (z.B. ein *Convolutional Neural Network*, CNN) die Extremitäten des Werkers. Sobald eine Hand oder Teile davon im Sichtfeld erfasst werden, wechselt das System in den definierten Sicherheitszustand. Diese Methode ist aufwendiger zu programmieren und benötigt eine signifikant höhere Rechenleistung (Schnittstellen für GPU/NPU). Sie arbeitet jedoch im industriellen Umfeld wesentlich zuverlässiger und minimiert die Erkennung von Pseudoobjekten. Dennoch besteht das Restrisiko einer fehlerhaften oder verzögerten Klassifikation (Falsch-Negativ-Rate). Zudem muss die Auslegung gemäß DIN EN ISO 13855 zwingend von der maximalen menschlichen Handgeschwindigkeit von 1600 mm/s bis 2000 mm/s ausgehen. KI-Modelle müssen daher eine entsprechend hohe Bildwiederholrate (Frames per Second, fps) aufweisen, um diese Dynamik latenzfrei abzugreifen. Ein ungelöstes Problem bleibt die mathematische "Black-Box-Problematik" von Deep-Learning-Algorithmen, da diese schwer deterministisch zu verifizieren sind. Für sicherheitskritische Systeme nach ISO 13849-1 ist der lückenlose Nachweis einer fehlerfreien Entscheidung eine der größten aktuellen Forschungsherausforderungen. Zudem zeigt sich eine strikte Abhängigkeit von den Trainingsdatensätzen; weisen diese keine hinreichende Varianz bezüglich Hautfarben, Arbeitshandschuhen oder Handhaltungen auf, droht ein potenzieller Ausfall der Schutzfunktion.

Auf der sensorischen Ebene stehen sich physische und visuelle Systeme gegenüber. Die physische Sensorik realisiert die Raum- und Kontaktherausforderung über klassische Komponenten wie Laserschranken um den Sicherheitsbereich, kapazitive Sensorfelder (z.B. als sensitive Roboterhaut) oder induktive Felder, die auf die Annäherung von Körperteilen reagieren. Physische Sensoren sind kostengünstig und zeichnen sich durch eine einfache Problembehandlung aus. Zudem eignen sie sich hervorragend als diversitäre, komplementäre Redundanz in Kombination mit optischen Systemen, um die strengen

Anforderungen an die Ein-Fehler-Sicherheit nach DIN EN ISO 13849-1 zu erfüllen.

Als Nachteil erweist sich die anspruchsvolle Kalibrierung, da die Sensorik sensibel auf elektromagnetische Umweltstörungen (EMV) reagiert und oft spezialisierte Auswerteelektronik benötigt. Da physische Sensoren zudem keine Tiefen- oder Richtungsinformationen liefern, können sie weder die Annäherungsgeschwindigkeit noch die genaue Position im Raum bestimmen. Ein zusätzlicher Faktor ist der mechanische Verschleiß sowie der hohe Montage- und Verkabelungsaufwand im rauen Industriealltag. Die visuelle Sensorik nutzt Kamerasysteme (2D-Systeme mit Kantenextraktion oder 3D-Time-of-Flight-Sensoren mit Tiefendaten) zur volumetrischen Raumüberwachung. Visuelle Systeme bieten den Vorteil einer zuverlässigen Klassifizierung spezieller Geometrien und erlauben Mehrfachanwendungen, indem dieselbe Hardware sowohl für Sicherheitsaufgaben als auch für bildverarbeitungsgestützte Greifprozesse (Pick-and-Place) genutzt werden kann. Eine laufende Neukalibrierung im Betrieb ist meist nicht notwendig.

Demgegenüber stehen hohe Anschaffungskosten und eine aufwendigere algorithmische Programmierung. Zudem reagieren optische Systeme empfindlich auf wechselnde Beleuchtungsverhältnisse, Schattenwurf oder Verschmutzungen der Linse. Normativ ist hierbei die Zertifizierung nach IEC 61496 zu beachten: Optische Kamerasysteme für Sicherheitsanwendungen müssen zwingend als berührungslos wirkende Schutzeinrichtungen (ESPE) nach IEC 61496-1 zertifiziert sein. Standardmäßige Industriekameras ohne inhärente Hardware-Redundanz und gegenseitige Prozessorüberwachung sind für sicherheitsgerichtete Abschaltungen unzulässig.

Als finale Aktuatorik stehen der abrupte Nothalt und das adaptive Abbremsen zur Auswahl. Ein Nothalt führt bei einer Schutzfeldverletzung zum sofortigen Stillsetzen des Roboterarms, geregelt über eine kontrollierte oder unkontrollierte Energietrennung (Kategorie 0/1 nach DIN EN 60204-1). Im Sicherheitskontext von Robotersystemen wird diese Reaktion gemäß ISO 10218-1 präzise als sicherheitsgerichteter überwachter Halt (Safety-rated monitored stop) definiert. Sie bietet die höchste Sicherheitsstufe zur Risikominimierung. Für kooperative Arbeitsräume ist diese Reaktion jedoch ineffizient, da jede Annäherung des Werkers zu einem vollständigen Produktionstopp führt. Die abrupten Momentenmaxima verursachen zudem einen massiven mechanischen Verschleiß der Robotergelenke und Getriebe. Darüber hinaus reduziert die erhebliche Wiederanlaufzeit des Gesamtsystems (Bremsen lösen, Synchronisation der Servoregler) die

Produktivität der Anlage signifikant. Eine wesentlich ergonomischere Lösung für die MRK stellt das Abbremsen des Roboterarms im Sinne des Speed and Separation Monitoring (SSM) dar. Der Manipulator reduziert seine Bahngeschwindigkeit in Abhängigkeit vom Abstand zum Menschen stufenweise oder kontinuierlich über vordefinierte Geschwindigkeitszonen. Diese Funktion der sicheren Geschwindigkeitsreduzierung (Safely-Limited Speed, SLS) sichert fließende Prozesse, minimiert den mechanischen Verschleiß und muss über sichere Antriebsregler (z.B. gemäß IEC 61800-5-2) validiert werden, um die funktionale Sicherheit zu garantieren.

Der Ansatz erfordert jedoch einen hohen softwareseitigen Aufwand für die Bildverarbeitung und verlangt bei Fehlfunktionen in der Regelung eine konsequente zweikanalige Überwachung. Zu beachten bleibt das regelungstechnische Restrisiko durch Trägheit: Aufgrund der Eigenmasse des Manipulators besitzt dieser auch während des Verzögerungsvorgangs eine hohe kinetische Energie. Das Sicherheitskonzept muss mathematisch garantieren, dass die Drosselung schnell genug erfolgt, bevor der Mensch die nächste kritische Distanzschwelle erreicht. Da der Roboter auch bei geringen Geschwindigkeiten hohe statische Kräfte aufbauen kann, muss das System so ausgelegt sein, dass ein Quetschen von Körperteilen zwischen dem Roboter und starren Umgebungsstrukturen physikalisch oder algorithmisch vollständig ausgeschlossen ist.

5.1.2 Gegenüberstellung spezialisiertes KI-Modell und einfacher Objekterkennungsalgorithmus

6 Fazit & Ausblick

7 Anhang

Literaturverzeichnis

- [1] Fraunhofer-Institut für Fabrikbetrieb und -automatisierung IFF, “Projektions- und kamerabasierte sicherheitstechnologien,” <https://www.iff.fraunhofer.de/de/geschaeftsbereiche/robotersysteme/projektions-und-kamerabasierte-technologien.html>, 2026, abgerufen am: 11. Juni 2026.
- [2] M. Bdiwi, M. Pfeifer, and A. Sterzing, “A new strategy for ensuring human safety during various levels of interaction with industrial robots,” *CIRP Annals*, vol. 66, no. 1, pp. 453–456, 2017.
- [3] Microsoft Developers. Task parallel library tpl. [Online; Zugegriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- [4] S. Cleary, *Concurrency in CSharp Cookbook*, 2nd ed. O'Reilly Media, Inc., 2019.
- [5] Microsoft Developers. asynchrone programmiermuster. [Online; Zugegriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/asynchronous-programming-patterns/>

Abkürzungsverzeichnis

CLR	Common Language Runtime
CMOS	Complementary Metal-Oxide-Semiconducto
Cobot	Collaborative Robot
CTS	Clear to Send
DFT	Diskrete Fouriertransformation
DH	Denavit Hartenberg
DIN	Deutsches Institut für Normung
DLL	Dynamic Link Library
DSR	Data Set Ready
DTR	Data Terminal Ready
DU	Drive Unit
EN	Europäische Norm
ER	Error Read
FG	Frame Ground
FOV	Field of View
fps	frames per second (Bilder pro Sekunde)
IPK	Institut für Produktionsanlagen und Konstruktionstechnik
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LED	Leuchtdiode
LIDAR	Light Detection and Ranging
MC	Move Continuous
ML	Maschinelles Lernen
MO	Move
ms	Millisekunde
Mutex	Mutual Exclusions
MVC	Model View Controller
NC	Not Connected
P/Invoke	Platform Invocation Services
PCL	Point Cloud Library
PL	Performance Level
RANSAC	Random Sample Consensus Algorithmus
RXD	Receive Data
ROR	Radius Outlier Removal
RS-232	Recommended Standard 232
SCARA	Selective Compliance Assembly Robot Arm
SOR	Statistical Outlier Removal

SP	Speed
SSM	Speed and Separation Monitoring
TCP	Tool Center Point
TCP/IP	Transmission Control Protocol/Internet Protocol
ToF	Time of Flight
TXD	Transmit Data
WH	Where
2D	Zweidimensional
3D	Dreidimensional

Abbildungsverzeichnis

2.1	Problematik von blockierenden Abläufen	4
3.1	Sequenzielle Abfolge an Befehlen (Eigene Darstellung) . .	7
4.1	Gegenüberstellung von blockierenden synchronen Abläufen und dem ressourceneffizienten, asynchronen Task-basierten Muster (TAP) bei E/A-Operationen.	10

Erklärungen zur Arbeit und Hilfsmitteln

Verwendete Hilfsmittel

Für die Erstellung dieser Studienarbeit wurden folgende Tools eingesetzt:

- **Hardware:** SICK Visionary-T Mini-CX, Mitsubishi Movemaster EX-RV M1, uEye Industriekamera
- **Software:** OpenCV Bibliothek, Visual Studio (C++ und C#), RoboETH, drawIO (Erstellung von Grafiken), texStudio, uEye-Cockpit und IDS Kameramanager, GitHub (Kollaboratives Arbeiten und Versionsverwaltung der LaTeX-Vorlage).

KI-Tools und relevanter Einsatzbereich

Dies erfolgt im Einklang mit der entsprechenden Verordnung der DHBW (Weblink).

Tool	Art der Nutzung	Abschnitt
Google Gemini AI Student Pro License	Formatierung von LaTeX-Code, Erstellung von Blankovorlagen für Tabellen und Listen	gesamte Arbeit
Google Gemini AI Student Pro License: Deep Search-Feature	Tiefgreifende Analyse des Datenblatts des Movemaster EX RV-M1: Finden von Abschlusszeichen im RS232-String, Pin-Belegung für Hardwaregestütztes Acknowledgement über RS232, Physikalische Signalanalyse für die Implementierung einer Flow-Control; Ergebnis der "Deep Search" kann jederzeit als PDF-Dokument bereitgestellt werden	4.3.2
DeepL AI Übersetzer	Formulierung des englischsprachigen Abstracts	Abstract

Aufteilung der Kapitel

Die Erstellung der vorliegenden Studienarbeit erfolgte in gemeinsamer Kooperation. In der folgenden Tabelle ist dokumentiert, welcher Autor die primäre Urheberschaft für die jeweiligen Kapitel bzw. Abschnitte trägt:

Kapitel / Abschnitt	Kapitelnummer	Autor
Einleitung	1	Gemeinsame Bearbeitung
Robotik	??	Luis Magnus Thiel
Industrielle Bildverarbeitung	??	Adrian Sommer
Grundlagen der Informatik	??	Luis Magnus Thiel
Hardware	??	Luis Magnus Thiel
Software	??	Adrian Sommer
Messreihe	??	Luis Magnus Thiel
Optimierung der Softwarestruktur	??	Luis Magnus Thiel
Optimierung der Bildgewinnung	??	Luis Magnus Thiel
Optimierung und Verbesserungsmöglichkeiten an der Software	??	Adrian Sommer
Normative Anforderungen	??	Gemeinsame Bearbeitung
Umsetzbare Sicherheitsanforderungen	??	Gemeinsame Bearbeitung
Ansatz Geschwindigkeits- und Abstandsüberwachung	??	Adrian Sommer
Fazit und Ausblick	??	Gemeinsame Bearbeitung

Hiermit bestätigen wir, Luis Magnus Thiel und Adrian Sommer,
dass die oben gemachten Angaben der Wahrheit entsprechen.

Horb, den 15. Juni 2026



Ort, Datum

Unterschrift (Luis Magnus Thiel)

Horb, den 15. Juni 2026



Ort, Datum

Unterschrift (Adrian Sommer)

Anhang

Inhalt des digitalen Anhangs

- Anleitung zur Programmaufsetzung (Luis Thiel)
- gesamter LaTeX-Ordner